

Bayesian Handwritten Digit Recognition

Last-Layer Laplace Inference with an Exact Conditional Hessian

Paco, Sou Chak Tong
Student ID: U-C3-2652-3
CISC3024 — Pattern Recognition
AI Assignment #2

8 October 2026

Abstract

This coursework implements a small-scale last-layer Laplace classifier motivated by *Laplace Redux* (NeurIPS 2021). A learned image representation is frozen, a Gaussian prior is placed over its linear classification head, and an exact conditional softmax Hessian defines a Gaussian approximation to the posterior. Predictions average class probabilities over 512 sampled heads. The study uses 1,797 handwritten digit images, a fixed stratified split of 1,077/360/360 images, and three independent neural initialization seeds. Validation selects checkpoints and prior precision before final test prediction. Laplace achieves 97.69% mean accuracy and 0.151 mean negative log-likelihood (NLL). A separately selected maximum a posteriori (MAP) classifier has lower NLL (0.099). A frozen-prior ablation confirms that posterior averaging reduces probability quality here, while dropping posterior correlations reduces it further. Thus Bayesian inference provides explicit uncertainty, but does not guarantee better calibration. The AI agent performed research, programming, experiments and report drafting; the personal reflection remains student-authored.

Scope. This is an independently coded, educational adaptation of a recent practical Bayesian framework, not a reproduction of its published benchmark results. All numerical results come from this project's saved outputs. Public publication is deliberately deferred to Prompt 03.

Contents

1	How AI Found the Algorithm	2
2	Algorithm Description	3
3	How AI Implemented the Algorithm	5
4	Experiment Settings and Results	6
5	What I Have Learnt from This AI Assignment	13
6	Webpage Link of the Source Codes	13

1 How AI Found the Algorithm

1.1 Instructions, search and verification

The instructor’s one-page assignment requests a recent classification or recognition algorithm based on probability or Bayes theory and allows AI to perform the technical work. It requires six report items and prohibits the student from writing the code. The agent read the full PDF and rendered it to verify the text. The initial chat was attached to the older denoising project; the requested Assignment 2 files were found in the sibling **AI Assignment 2** directory, where all new work was performed.

The project’s `AGENTS.md` supplied persistent constraints: substantive Bayesian inference, isolated held-out data, truthful execution evidence and a student-authored reflection. The user invoked `01_RESEARCH_IMPLEMENT_REPORT.md`, which decomposed work into literature choice, implementation, execution and report preparation. These were instructions actually received, rather than a reconstruction of an earlier dialogue.

On 8 October 2026, searches included *Laplace Redux Effortless Bayesian Deep Learning NeurIPS 2021 Daxberger*, *Posterior Network uncertainty estimation without OOD samples NeurIPS 2020 Natural Posterior Network 2021*, and *2022 Bayesian neural network classification variational inference last layer uncertainty paper*. Primary proceedings, author manuscripts and dataset documentation were opened. OpenReview forum pages returned a verification challenge; accessible PDFs and arXiv records supplied candidate metadata. This bounded search is a targeted narrative review, not an exhaustive systematic review. Query strings and source links are retained in `docs/research.md`.

1.2 Three substantive candidates

Method	Publication	Defining probability mechanism	Feasibility and main risk
Laplace Redux	NeurIPS 2021	Gaussian approximation around MAP; posterior integration	Post-hoc head inference is tractable; a local Gaussian can misrepresent uncertainty.
Natural Posterior Network	ICLR 2022	Density-dependent conjugate updates for exponential-family targets	Learned features and normalizing flows; density training adds complexity and failure modes.
Variational Bayesian Last Layers	ICLR 2024	Deterministic variational formulation for Bayesian output layers	Efficient last-layer training; its variational objective adds implementation work.

Table 1: Candidate methods verified from primary sources [1, 2, 3]. All address probabilistic prediction and can be evaluated on image recognition.

The selected framework supports last-layer inference, allowing inspection of a complete covariance matrix and independent verification of its derivatives. The alternatives are suitable, more recent methods; the choice was driven by mathematical transparency and reliable local execution. The 2021 publication is at the older end of Prompt 01’s approximately 2021–2026 range. Its contribution is a practical synthesis, implementation library and evaluation of Laplace variants; neither Bayes’ theorem nor the Laplace approximation was invented in 2021. Earlier Bayesian neural-network work is acknowledged [4]. No state-of-the-art claim is made for this small dataset.

1.3 Actual skill contributions

The agent consulted literature-review, experimental-design, scikit-learn and scientific-visualization skills for provenance, matched comparisons, splitting, metric conventions and legible plots. Machine-learning and PyTorch guidance informed deterministic training, device handling and checkpoint checks. The PDF skill supplied extraction and page inspection practices; the LaTeX compile plugin supplied compilation. The Scientific Agent Skills reference acknowledges skills that materially contributed [8]. Paid schematic generators, external manuscript review and multi-agent review were not used.

2 Algorithm Description

2.1 Task, notation and conditional model

An image is an 8×8 grayscale array with integer acquisition values from 0 to 16. Flattening gives $x \in [0, 1]^{64}$ after division by the fixed maximum 16. Let $y \in \{0, \dots, 9\}$ be its digit label, $K = 10$, and $\mathcal{D} = \{(x_n, y_n)\}_{n=1}^N$ the training partition. A deterministic neural representation is

$$h_\phi(x) = \tanh(Ax + a) \in \mathbb{R}^{32}, \quad f_\phi(x) = \begin{bmatrix} h_\phi(x) \\ 1 \end{bmatrix} \in \mathbb{R}^D, \quad D = 33. \quad (1)$$

The appended one makes bias parameters part of the same algebra as weights. A head $W \in \mathbb{R}^{K \times D}$ produces logits $z_k = w_k^\top f_\phi(x)$. After training the representation, ϕ is frozen. Bayesian inference is *conditional on this learned representation*; it does not infer a distribution over all network weights.

Symbol	Meaning
$N = 1077$	Training images
$K = 10, D = 33$	Classes and augmented feature dimension
$\theta = \text{vec}_{\text{row}}(W) \in \mathbb{R}^{330}$	Class-major head parameter vector
$\alpha > 0$	Isotropic Gaussian prior precision
H, Σ	Posterior precision Hessian and covariance
$S = 512$	Monte Carlo weight draws per prediction run
$q(\theta)$	Gaussian approximation to the conditional posterior

Table 2: Notation matched to array layouts in `laplace.py`.

2.2 Likelihood, prior and posterior

The defining conditional likelihood is categorical:

$$p(y = k \mid x, W, \phi) = p_k(x; W) = \frac{\exp(w_k^\top f_\phi(x))}{\sum_{j=0}^{K-1} \exp(w_j^\top f_\phi(x))}. \quad (2)$$

Training observations are modeled as conditionally independent. An isotropic prior is imposed on *every* head parameter, including biases:

$$p(\theta \mid \alpha) = \left(\frac{\alpha}{2\pi}\right)^{KD/2} \exp\left(-\frac{\alpha}{2}\|\theta\|_2^2\right). \quad (3)$$

Bayes' rule gives

$$p(\theta \mid \mathcal{D}, \phi, \alpha) = \frac{p(\mathcal{D} \mid \theta, \phi)p(\theta \mid \alpha)}{\int p(\mathcal{D} \mid u, \phi)p(u \mid \alpha) du}. \quad (4)$$

For fixed α and ϕ , constants independent of θ can be discarded. The negative log-posterior objective is therefore

$$L(\theta) = \sum_{n=1}^N \left[\log \sum_j \exp(w_j^\top f_n) - w_{y_n}^\top f_n \right] + \frac{\alpha}{2}\|\theta\|_2^2. \quad (5)$$

The implementation uses stable `logsumexp` and sums the likelihood. A mean loss with the same numerical α would change the prior's relative strength by a factor of N . Feature training uses a separate mean cross-entropy and regularizer; its optimizer decay is not identified with this conditional head prior.

2.3 MAP estimation and exact curvature

For fixed features, the objective is convex. A strictly positive prior precision makes it strictly convex, with a unique head optimum $\hat{\theta} = \arg \min_\theta L(\theta)$. Let $t_{nk} = \mathbb{1}\{y_n = k\}$. Differentiating the normalizer gives $\partial \log \sum_j e^{z_j} / \partial z_k = p_{nk}$, hence

$$\frac{\partial L}{\partial w_k} = \sum_{n=1}^N (p_{nk} - t_{nk}) f_n + \alpha w_k. \quad (6)$$

L-BFGS minimizes this objective using an analytic gradient. Every run checks the optimizer status and requires the maximum absolute summed-objective gradient component to be below 10^{-3} . Recorded candidate gradients are much smaller.

Differentiating softmax gives $\partial p_k / \partial z_l = p_k(\delta_{kl} - p_l)$. Thus the exact Hessian entries, in the class-major ordering used by the code, are

$$H_{(k,a),(l,b)} = \sum_{n=1}^N p_{nk}(\delta_{kl} - p_{nl})f_{na}f_{nb} + \alpha\delta_{kl}\delta_{ab}. \quad (7)$$

Equivalently, with $B_n = \text{diag}(p_n) - p_n p_n^\top$,

$$H = \sum_{n=1}^N B_n \otimes (f_n f_n^\top) + \alpha I_{KD}. \quad (8)$$

This is the *exact conditional* Hessian for linear logits. It is not an exact full-network Hessian: nonlinear feature parameters are held fixed. A generalized Gauss–Newton expression coincides with this Hessian in the conditional linear-logit case.

For any $v \in \mathbb{R}^K$, $v^\top B_n v = \sum_k p_{nk} v_k^2 - (\sum_k p_{nk} v_k)^2 \geq 0$: it is a variance under a categorical distribution. Thus $H \succeq \alpha I$ and Cholesky factorization is well defined. At $\alpha = 0$, a common-logit shift direction is unidentifiable and curvature is singular. A proper prior resolves this issue. Nonpositive precisions are rejected rather than hidden by arbitrary damping.

2.4 Local Gaussian approximation

Expand the negative log-posterior near its optimum:

$$L(\theta) \approx L(\hat{\theta}) + \underbrace{\nabla L(\hat{\theta})^\top (\theta - \hat{\theta})}_{\approx 0} + \frac{1}{2}(\theta - \hat{\theta})^\top H(\theta - \hat{\theta}). \quad (9)$$

Exponentiating its negative and normalizing yields

$$q(\theta) = \mathcal{N}(\hat{\theta}, H^{-1}), \quad \Sigma = H^{-1}. \quad (10)$$

A Gaussian prior is not conjugate to a categorical softmax likelihood; the true posterior is generally not Gaussian. The second-order expansion is an approximation even when its Hessian is exact. The code computes the full 330×330 precision matrix in double precision. No diagonal or Kronecker curvature approximation is used for the primary method. The small problem makes its full covariance practical and indexing testable.

Illustrative example. In binary logistic regression with one coefficient, one observation, feature $f = 1$, probability $p = 1/2$ and $\alpha = 1$, scalar curvature is $H = p(1-p)f^2 + \alpha = 1.25$ and local variance is 0.8. This is an algebraic illustration, not an experimental metric. Increasing α to 10 gives variance $1/10.25$ if likelihood curvature is held fixed. When refitting, the optimum and likelihood curvature also change.

2.5 Posterior predictive classification

A Bayesian prediction integrates the uncertain head:

$$\bar{p}_k(x) = \int p_k(x; \theta) q(\theta) d\theta \approx \frac{1}{S} \sum_{s=1}^S p_k(x; \theta^{(s)}), \quad \hat{y}(x) = \arg \max_k \bar{p}_k(x). \quad (11)$$

Let $H = LL^\top$ with lower-triangular L . For $\epsilon^{(s)} \sim \mathcal{N}(0, I)$, sampling uses a triangular solve:

$$\theta^{(s)} = \hat{\theta} + L^{-\top} \epsilon^{(s)}, \quad \text{Cov}(L^{-\top} \epsilon) = L^{-\top} L^{-1} = H^{-1}. \quad (12)$$

The transpose matters: $L^{-1}\epsilon$ generally gives the wrong covariance. A simulation test checks sample covariance against a known non-diagonal precision inverse. The same 512 sampled heads predict all images within one call, representing global parameter uncertainty.

Validation draws use seed $r+1000$ and test draws $r+2000$ for initialization seed $r \in \{11, 22, 33\}$. Averaged class probabilities remain normalized. In general $\mathbb{E}[\text{softmax}(z)] \neq \text{softmax}(\mathbb{E}[z])$, so MAP and Laplace may disagree on probabilities even when their class predictions agree.

2.6 Uncertainty and interpretation

Predictive entropy summarizes total uncertainty:

$$\mathcal{H}(\bar{p}) = - \sum_k \bar{p}_k \log \bar{p}_k. \quad (13)$$

Posterior disagreement is quantified by

$$\hat{I}(y, \theta \mid x, \mathcal{D}) = \mathcal{H}(\bar{p}) - \frac{1}{S} \sum_s \mathcal{H}(p(\cdot \mid x, \theta^{(s)})). \quad (14)$$

Concavity of entropy makes this nonnegative up to numerical round-off. It describes the fitted *conditional approximate posterior*; it is not complete uncertainty about the whole network. No OOD detection benchmark or coverage guarantee is claimed.

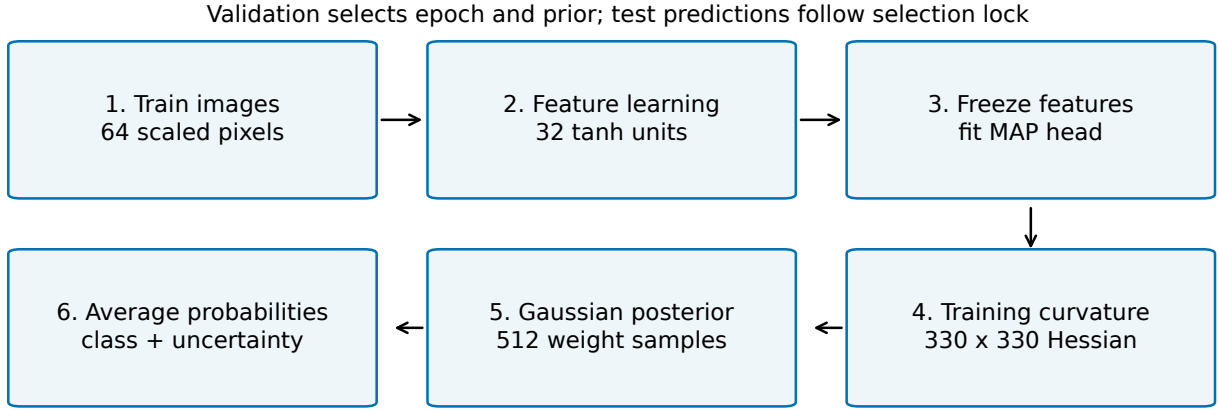


Figure 1: Implemented inference flow. Training supplies features and curvature; validation selects checkpoints and α . Test predictions follow the selection record. Numbers and arrows specify the traversal order.

2.7 Cost and limitations

With $P = KD = 330$, full Hessian storage is $O(P^2)$ and Cholesky factorization is $O(P^3)$. Dense curvature accumulation costs $O(NK^2D^2)$ by direct formulation, sampling costs $O(SP^2)$, and logits cost $O(SMKD)$ for M evaluation images. Feature extraction is additional. These costs scale poorly for very wide heads; full curvature is used here for auditability.

The posterior is local and depends on the feature point estimate, Gaussian prior and model specification. Monte Carlo probabilities have sampling error. The model omits representation uncertainty. Selecting among three priors by validation is not marginalization over α or marginal-likelihood optimization. These project choices are distinct from the literature framework.

3 How AI Implemented the Algorithm

3.1 Modules and equation-to-code mapping

All experiment and reporting code was written by the AI agent. The authoritative workflow is modular Python, without a notebook dependency. Configuration, training, inference, evaluation and presentation are separated.

File	Responsibility and mathematical connection
<code>data.py</code>	Images, fixed-range scaling, stratified split indices.
<code>features.py</code>	h_ϕ , deterministic training, validation checkpoint.
<code>laplace.py</code>	Objective, gradient, Eq. 7, Cholesky draws.
<code>metrics.py</code>	Classification metrics, NLL, Brier, calibration bins.
<code>experiment.py</code>	Candidates, selection lock, final test predictions.
<code>figures.py</code>	Eight figures derived from saved evidence.
<code>audit_results.py</code>	Metrics, checkpoint reload and full-run verification.
<code>build_report.py</code>	Numerical LaTeX tables generated from JSON.

Table 3: Modules reside under `src/bayes_digits/`, scripts under `scripts/`.

The head implementation is original project code, not a call to the paper’s `laplace` library. It implements one supported inference variant, not every library feature or the paper’s benchmarks.

3.2 Executed pipeline pseudocode

1. Load data, fix stratified indices, save configuration and dataset hash.
2. Fit Gaussian NB smoothing candidates on training images; select by validation NLL.
3. For each seed, train the image network for up to 120 epochs; retain the checkpoint with minimum validation cross-entropy.
4. Freeze features. For each $\alpha \in \{0.1, 1, 10\}$, refit a MAP head, check optimizer convergence and compute training-only posterior curvature.
5. Evaluate MAP and Laplace candidates on validation images; choose their priors separately by minimum validation NLL.
6. Finish all seeds and write `selection_locked.json`. Only then transform test images and predict.
7. Evaluate primary models and the two predeclared frozen-prior ablations. Save test probabilities, indices, labels, confusion matrices and metrics.
8. Generate plots and LaTeX tables; reload checkpoints and rerun the pipeline.

3.3 Checks, repairs and evidence

Seven tests passed. They verify split coverage/disjointness, analytic derivatives against independent PyTorch autograd, precision-to-covariance sampling, normalization and serialization, known metric values and bin boundaries, deterministic tiny-batch optimization, and rejection of invalid priors. The tiny-batch test achieves perfect training classification and loss below 0.02 on 20 images; repeated seeded feature training matches exactly.

The first test run failed while creating a Windows system temporary directory; the serialization test had not executed. Moving pytest temporary files inside the project resolved the access issue. A small end-to-end run with three feature epochs executed before the full run. It selected its own models with validation and evaluated after its selection lock; its test results were not used to change the already specified full configuration. The audit then reloaded all primary checkpoints, regenerated recorded metrics, and repeated the full pipeline. All 18 saved arrays matched bit for bit in this environment.

4 Experiment Settings and Results

4.1 Dataset provenance and protocol

The data are the `sklearn.datasets.load_digits` copy of the UCI optical handwritten digits dataset’s 1,797-image *test subset* [5, 6]. The full UCI collection has a separate larger training file, which this project does not use. These experiments repartition the bundled subset; they are not results on UCI’s original official protocol. UCI records CC BY 4.0 usage conditions and a dataset DOI.

The images are available offline through scikit-learn, and the dataset is small enough to verify full posterior matrices. Recognition uses flattened images without augmentation, pretrained weights, PCA or fitted standardization. Division by 16 follows the measurement range and fits no preprocessing parameters on held-out images.

A stratified split holds out 20% of all images with random state 3024. A second stratified split holds out 25% of the remainder using the same state. Integer rounding gives 1,077 training, 360 validation and 360 test images. Indices and per-class counts are saved. Test support ranges from 35 to 37 images per class. Figure 2 uses those saved indices.

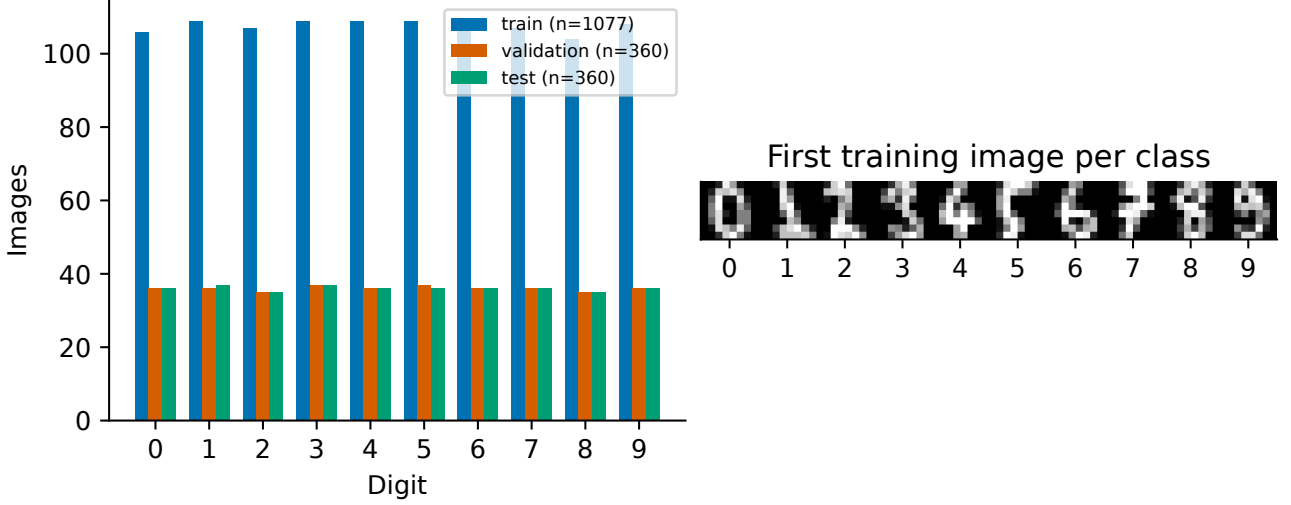


Figure 2: Class counts by split and the first training image of each digit. The range and nearest-neighbor display are fixed; examples are chosen deterministically.

The evaluation unit is an image in this fixed subset. Writer identifiers are not provided by the bundled loader, so writer-disjoint splitting is not demonstrated. A new-writer deployment claim is unsupported. Three seeds describe optimization variability conditional on one split; they do not turn 360 held-out images into 1,080 independent test examples.

4.2 Research questions and controls

The pre-execution questions were whether conditional last-layer inference retains useful accuracy, whether posterior averaging improves probability scores over a matched point predictor, and whether posterior correlations matter. The frozen-prior MAP ablation isolates averaging at the same learned representation and prior. The diagonal ablation isolates curvature structure with the same mean, selected prior and sample count.

Gaussian NB is a simple statistical baseline with conditionally independent Gaussian pixels and empirically estimated class priors. Spatial pixel correlations challenge its assumptions. The separately tuned MAP head is a discriminative comparison. All methods use identical partitions and a validation-NLL selection rule. Equal architecture or search difficulty between NB and the neural feature model is not assumed.

4.3 Exact configuration and environment

Setting	Executed value
Input, classes	64 pixels in $[0, 1]$, ten classes
Feature network	Linear $64 \rightarrow 32$, tanh, linear $32 \rightarrow 10$ training head
Neural parameter count	2,410 including biases
Initialization seeds; split seed	11, 22, 33; 3024
Feature training	Adam, full training batch, learning rate 0.01
Feature decay; epoch budget	10^{-4} ; 120 epochs
Checkpoint criterion	Minimum validation mean cross-entropy
Conditional head inference	SciPy L-BFGS-B, float64, max 2,000 iterations
Stopping options	ftol 10^{-14} , gtol 10^{-7} ; gradient check $< 10^{-3}$
Prior precision grid	0.1, 1, 10; biases included
Gaussian NB smoothing grid	10^{-9} , 10^{-3} , 10^{-1}
Posterior prediction	512 samples, full 330×330 Hessian
Calibration bins	Ten equal-width top-label confidence bins
Actual device; CPU threads	CPU; four PyTorch threads
Python; PyTorch	3.12.14; 2.11.0+cu128
NumPy; SciPy; scikit-learn	2.5.2; 1.18.1; 1.9.1
Operating system	Windows 11, build 26300
Detected GPU	NVIDIA GeForce RTX 5060 Ti; available but unused

Table 4: Configuration and observed environment. GPU availability does not imply GPU execution. Complete provenance is in `environment.json`.

CPU computation was chosen because this small full-batch problem and its dense conditional Hessian are inexpensive. The original training/selection/evaluation command took 5.97 seconds, excluding setup, tests, plotting and compilation. This is one observed wall-clock duration, not a controlled hardware benchmark. Existing PyTorch/scientific packages were reused read-only through a local environment path; missing scikit-learn dependencies were installed in the new Assignment 2 environment. README also gives a standalone setup recipe.

Seed	Feature epoch	MAP α	LA α	LA validation NLL
11	120	0.1	1	0.1516
22	120	0.1	1	0.1519
33	120	1	1	0.1648

Table 5: Choices locked before full-run test predictions. All feature checkpoints were selected at epoch 120. Laplace chose $\alpha = 1$ for every seed. Gaussian NB selected smoothing 0.1.

The validation minimum at the budget boundary indicates that loss was still improving over this range. The prespecified budget is reported rather than increased in response to test results. Each prior candidate’s validation NLL, iterations and gradient residual is preserved. Full test probabilities permit later audits without changing model choices.

4.4 Evidence preservation

The saved configuration, split indices, dataset hash, selection lock and environment record identify the protocol. `metrics.json`, `predictions.npz` and `scalar_metrics.csv` preserve numerical evidence. Model files store the selected representation and conditional posterior. Plot and table generators consume these saved files rather than embedded hand-typed measurements.

4.5 Metric definitions and overall results

For $M = 360$, accuracy is the proportion of correct argmax labels. Class precision is $TP/(TP + FP)$, recall is $TP/(TP + FN)$, and $F_1 = 2PR/(P + R)$, with zero when the relevant denominator vanishes. Macro metrics

average the ten class-specific values. Laplace probability metrics use posterior-averaged probabilities:

$$\text{NLL} = -\frac{1}{M} \sum_{i=1}^M \log \bar{p}_{y_i}(x_i), \quad (15)$$

$$\text{Brier} = \frac{1}{M} \sum_{i=1}^M \sum_{k=0}^{K-1} (\bar{p}_k(x_i) - \mathbb{I}\{y_i = k\})^2. \quad (16)$$

Logs are natural, so NLL is in nats. NLL clips probabilities below 10^{-15} . Brier is summed over classes, not divided by ten; its range is $[0, 2]$. NLL of averaged probabilities differs from average sampled-head NLL.

For top-label calibration, confidence is $c_i = \max_k \bar{p}_k(x_i)$. Let $B_b = \{i : b/10 \leq c_i < (b+1)/10\}$, with the final bin also including $c_i = 1$. Then

$$\text{ECE} = \sum_{b=0}^9 \frac{|B_b|}{M} |\text{accuracy}(B_b) - \text{confidence}(B_b)|. \quad (17)$$

Empty bins contribute zero. This is a usual top-label calibration convention [7]; ECE depends on binning and is not a proper scoring rule. NLL and Brier carry more weight in evaluating probability quality.

Method	Accuracy (%)	Macro-F1	NLL	Brier
Gaussian NB	91.94	0.920	0.630	0.134
MAP	97.78 ± 0.28	0.978 ± 0.003	0.099 ± 0.003	0.040 ± 0.002
Laplace	97.69 ± 0.16	0.977 ± 0.002	0.151 ± 0.003	0.055 ± 0.001

Table 6: Neural results are mean $\pm$ sample SD across three seeds. The deterministic NB baseline was evaluated once. SD describes seed variability on a shared test set, not a confidence interval.

Method	Macro-precision	Macro-recall	ECE
Gaussian NB	0.924	0.920	0.063
MAP	0.978 ± 0.002	0.978 ± 0.003	0.017 ± 0.008
Laplace	0.977 ± 0.001	0.977 ± 0.002	0.080 ± 0.003

Table 7: Additional classification and calibration metrics. Lower ECE is better under the specified ten-bin estimator.

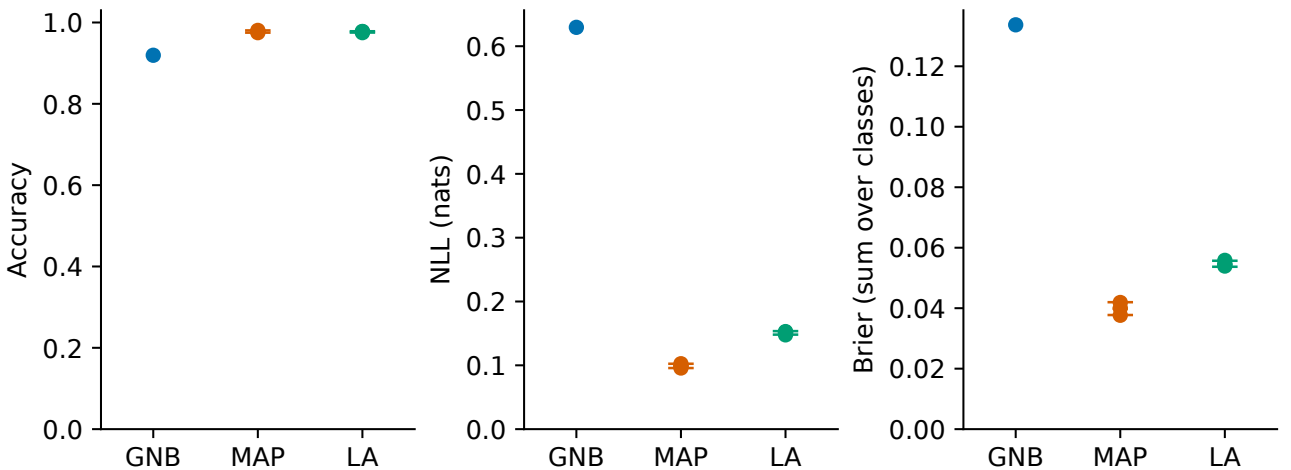


Figure 3: Per-run primary results and sample SD. Axes start at zero. GNB has one observation; MAP and LA have three, using the same 360 test images.

The neural methods recognize digits more accurately than NB. MAP and Laplace have similar accuracy, but MAP has lower NLL and Brier; Laplace ECE is higher. The experiment therefore does not support improved probability quality from this approximation. Accuracy alone would miss that result. Small accuracy differences involve only a few images. No significance test or statistical equivalence claim is made.

4.6 Class-level results and feature training

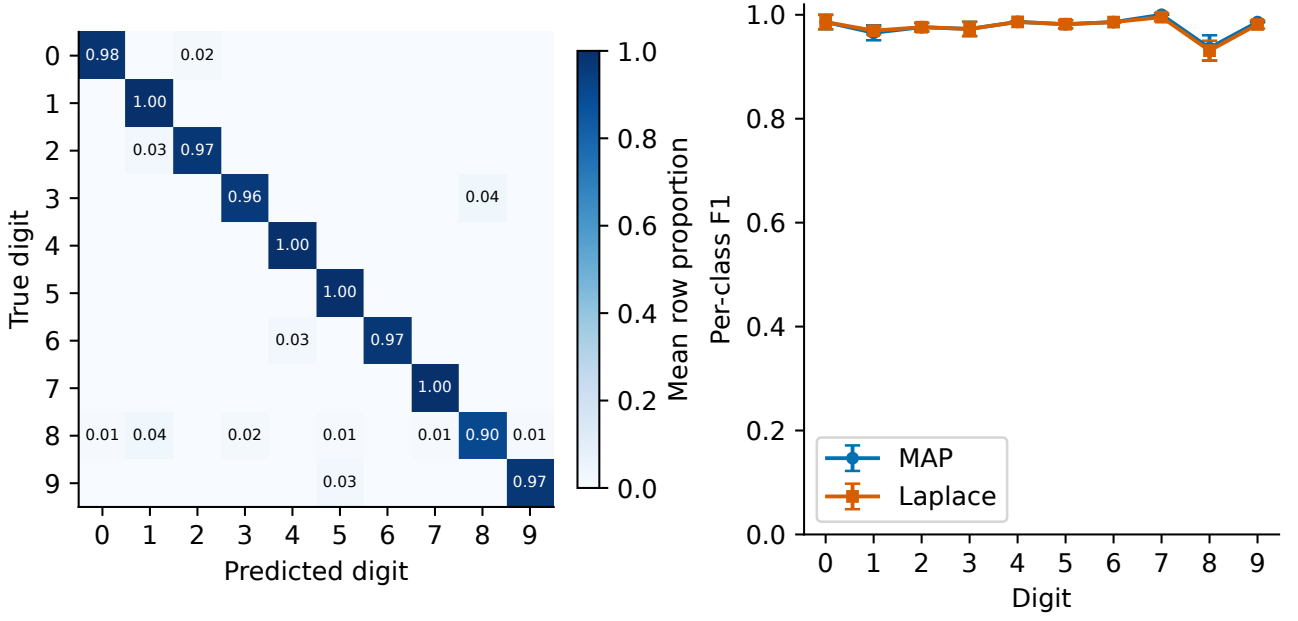


Figure 4: Laplace confusion is normalized by true-class support within each seed, then averaged. The right panel shows class F1 means and seed SD. There are 360 unique test images. Entries below 0.005 are unlabeled.

Digit	Test support	Precision	Recall	F1
0	36	0.991	0.981	0.986
1	37	0.941	1.000	0.969
2	35	0.981	0.971	0.976
3	37	0.982	0.964	0.973
4	36	0.973	1.000	0.986
5	36	0.964	1.000	0.982
6	36	1.000	0.972	0.986
7	36	0.991	1.000	0.995
8	35	0.960	0.905	0.931
9	36	0.991	0.972	0.981

Table 8: Mean Laplace per-class metrics across three seeds. Support counts unique images, not three times that number.

Digit 8 has the weakest class F1 (0.931), with recall 0.905. Its averaged mistakes are spread across several labels, including 1 and 3. Digit 1 has perfect recall but lower precision because other digits are sometimes assigned to it. Thus recall alone does not establish reliable predictions of a label.

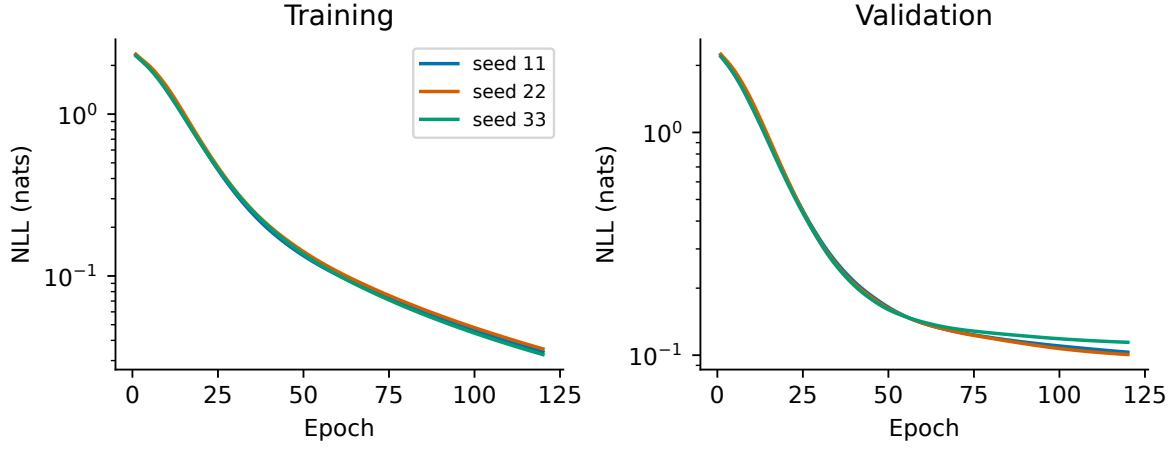


Figure 5: Recorded feature-training and validation mean NLL on logarithmic axes. These are losses of the original training head, before the conditional MAP head is refitted.

Both losses decrease through the prescribed budget. The curves do not prove asymptotic feature-network convergence. Conditional head optimization is a separate convex problem with checked gradient residuals; posterior inference has no stochastic training trajectory in this implementation.

4.7 Calibration, uncertainty and ablations

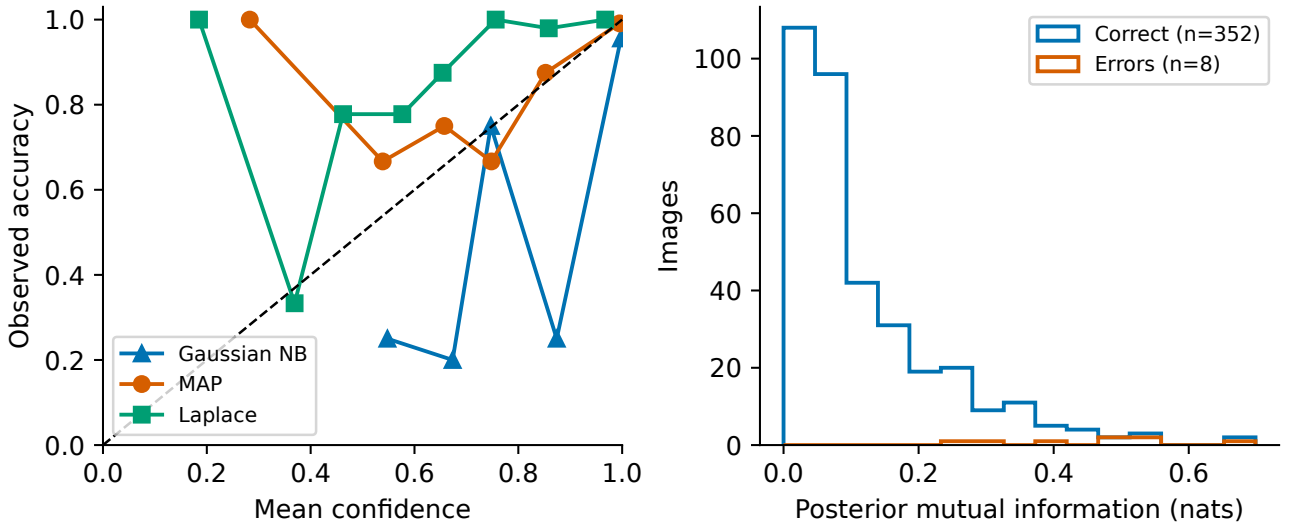


Figure 6: Reliability and posterior mutual information for prespecified seed 11. Ten equal-width confidence bins are used; sparse bins can be unstable. Histograms show counts of 352 correct and eight incorrect predictions.

Several populated Laplace reliability bins lie above the identity line: confidence is lower than observed success frequency. This agrees with higher ECE/NLL, although sparse bins and only eight mistakes limit conclusions. Many correct images have low posterior disagreement, while errors span a broader range; overlap prevents a guaranteed error detector. The representative seed was fixed before training.

Frozen-prior variant	Accuracy (%)	NLL	ECE
Matched MAP	97.69 ± 0.16	0.097 ± 0.002	0.025 ± 0.002
Laplace	97.69 ± 0.16	0.151 ± 0.003	0.080 ± 0.003
Diagonal Laplace	97.69 ± 0.42	0.229 ± 0.000	0.147 ± 0.005

Table 9: Features and Laplace-selected prior are held fixed. Matched MAP removes averaging; diagonal Laplace drops off-diagonal precision terms before inversion. This differs from taking the diagonal of the full covariance.

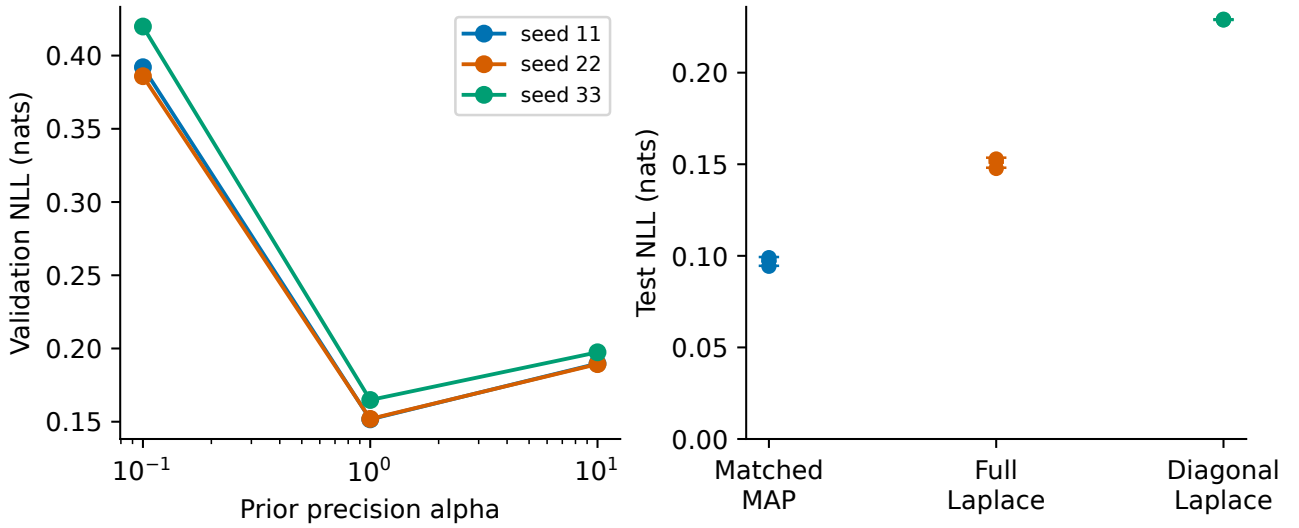


Figure 7: Left: validation-only prior sensitivity. Right: predeclared frozen-prior test NLL and seed SD. Test scores were not used to retune the prior.

Matched MAP has lower NLL than full Laplace with essentially identical aggregate accuracy, isolating the effect of averaging rather than a different prior. Diagonal Laplace increases NLL further. Keeping correlations improves this approximation relative to the diagonal variant, but not relative to the point estimate. Removing correlations can destroy cancellation of uncertain common-logit directions. This mathematical explanation is compatible with the results, not a separately measured causal decomposition.

Both $\alpha = 0.1$ and $\alpha = 10$ give worse Laplace validation NLL than $\alpha = 1$ for every seed. This demonstrates prior sensitivity within the specified grid, not a globally optimal precision or an empirical-Bayes result.

4.8 Failure cases, interpretation and limits

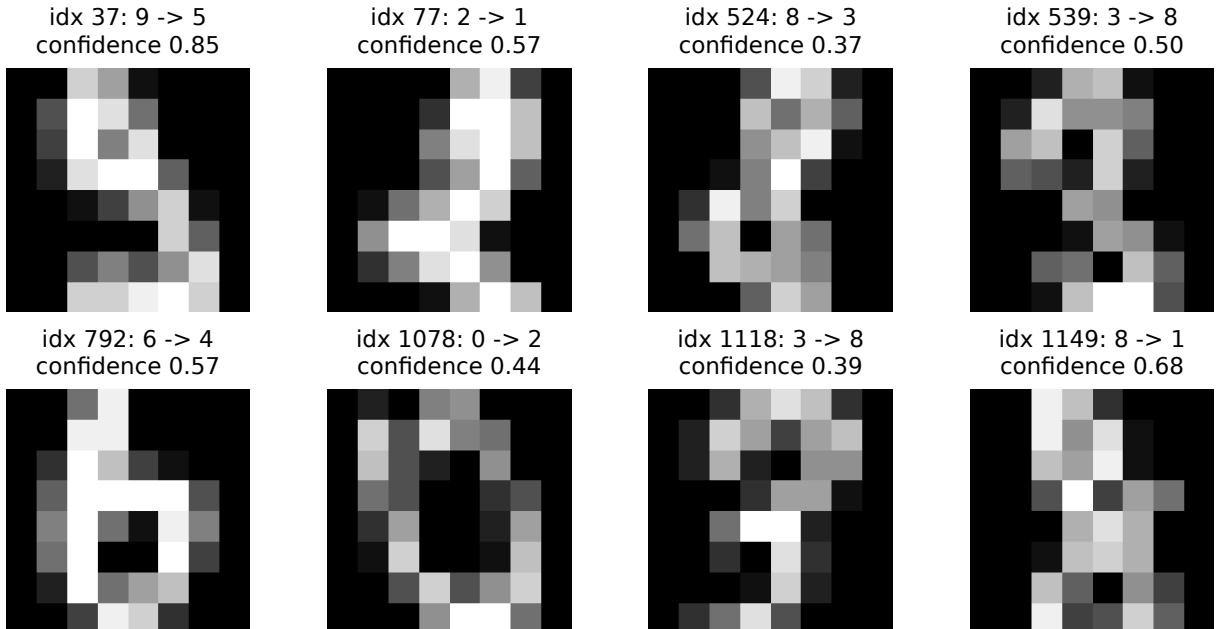


Figure 8: All eight Laplace errors for seed 11, ordered by original dataset index. Titles show true digit, prediction and maximum posterior probability. Native 8×8 pixels are displayed without smoothing.

Several mistakes involve ambiguous low-resolution strokes; that interpretation is descriptive, not a measured explanation of image quality. The 9-to-5 case has confidence 0.85, showing that a Bayesian predictor can still make a relatively confident mistake. The 8-to-3 case has much lower confidence. Selecting only favorable examples would conceal this variation.

Answers to the research questions. Conditional Laplace inference retains high recognition accuracy in this experiment. It does not improve the primary proper probability scores over independently selected MAP or the matched-prior MAP control. Full covariance improves NLL over diagonal precision, supporting attention to covariance structure rather than treating approximations as interchangeable.

Interpretation boundary. One small, nearly balanced dataset, one split, one architecture and three initialization seeds cannot establish a general ranking of Bayesian classifiers. The Gaussian is local and omits representation uncertainty. The prior grid is coarse and feature-scale dependent. Checkpoint and prior selection reuse the same validation partition; there is no nested validation estimate. Monte Carlo sampling error is not separately quantified. Writer independence, natural-image recognition, corruption and OOD detection were not tested.

Prospective extensions include a wider validation-selected prior grid, empirical-Bayes tuning, independent Monte Carlo repetitions, writer-separated data or a second dataset. They were not executed and do not justify revising the current conclusion using test outcomes. The negative calibration result belongs in the evidence.

Reproducibility. Source hashes bind the original run to its actual modules, even though no commit existed then. `audit.json` records metric regeneration, checkpoint reloads and an identical full rerun. Numerical tables read `summary.json` and `metrics.json`; figure provenance records inputs and dimensions. These checks establish local reproducibility under the observed environment, not bitwise portability across every platform or library version.

5 What I Have Learnt from This AI Assignment

STUDENT REFLECTION REQUIRED

This section is reserved for the student's own learning account. The AI agent has not supplied personal experiences or reflections.

6 Webpage Link of the Source Codes

Publication is deliberately deferred to Prompt 03, after quality review and the student's reflection. No public URL has been created for this Assignment 2 project during Prompt 01. The GitHub CLI check reported an invalid stored token; publication and wiki verification require functioning authenticated access.

The local source is in the **AI Assignment 2** directory. A new public repository must be used; Assignment 1's repository is unrelated. Wiki sources under `docs/wiki/` cover the method, protocol, results, reproduction and AI workflow. This is the actual publication status, not an invented link.

Local reproduction

After installing the pinned requirements, execute from the project root:

```
$env:PYTHONPATH = "src" # PowerShell
.venv\Scripts\python.exe -m pytest -q
.venv\Scripts\python.exe -m bayes_digits.experiment
.venv\Scripts\python.exe -m bayes_digits.figures --results results
.venv\Scripts\python.exe scripts/build_report.py
.venv\Scripts\python.exe scripts/audit_results.py
```

Compile `report/main.tex` with its tables and figures available. The executed compiler was Tectonic 0.17.0 via the installed LaTeX plugin. README includes standalone installation and compilation commands. The generated PDF is rendered and reviewed page by page.

References

- [1] E. Daxberger, A. Kristiadi, A. Immer, R. Eschenhagen, M. Bauer and P. Hennig. *Laplace Redux—Effortless Bayesian Deep Learning*. NeurIPS 34, 2021.
<https://papers.nips.cc/paper/2021/hash/a7c9585703d275249f30a088cebba0ad-Abstract.html>.

- [2] B. Charpentier, O. Borchert, D. Zügner, S. Geisler and S. Günnemann. *Natural Posterior Network: Deep Bayesian Uncertainty for Exponential Family Distributions*. ICLR, 2022. <https://arxiv.org/abs/2105.04471>.
- [3] J. Harrison, J. Willes and J. Snoek. *Variational Bayesian Last Layers*. ICLR, 2024. <https://arxiv.org/abs/2404.11599>.
- [4] D. J. C. MacKay. *Bayesian Interpolation*. Neural Computation 4(3):415–447, 1992. <https://doi.org/10.1162/neco.1992.4.3.415>.
- [5] E. Alpaydin and C. Kaynak. *Optical Recognition of Handwritten Digits*. UCI dataset, 1998. <https://doi.org/10.24432/C50P49>. CC BY 4.0.
- [6] Scikit-learn documentation. *load_digits*. Accessed 8 October 2026. https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_digits.html.
- [7] C. Guo, G. Pleiss, Y. Sun and K. Q. Weinberger. *On Calibration of Modern Neural Networks*. ICML, PMLR 70:1321–1330, 2017. <https://proceedings.mlr.press/v70/guo17a.html>.
- [8] T. Kassis, V. Agarwal, Y. He, D. Patel and A. M. Brueckner. *Scientific Agent Skills: A Library of Procedural Knowledge for Research Agents*. ArXiv:2609.00065, 2026; record revised 2 September 2026. <https://doi.org/10.48550/arXiv.2609.00065>.